Redis/Valkey Over RDMA
极致的高性能 KV 存储

皮振伟 - 2024/11

ByteDance字节跳动

Redis/Valkey 发展史

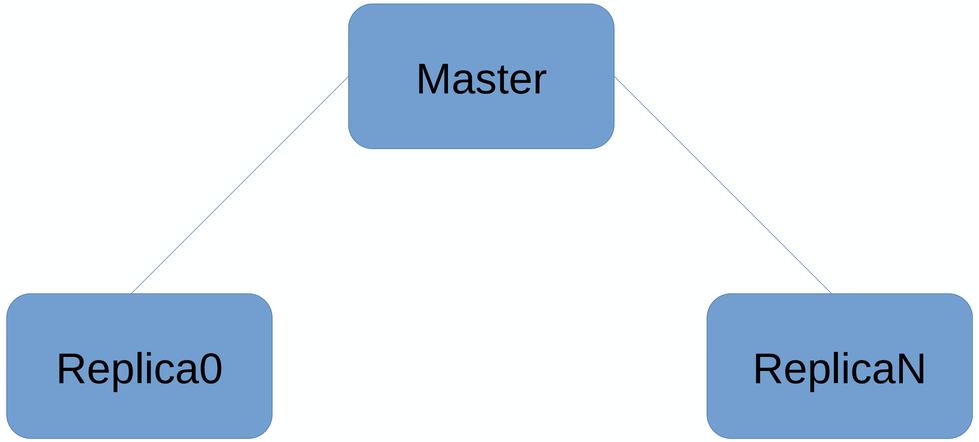
Redis/Valkey 发展史

- 2009 - Salvatore Sanfilippo (antirez) 最初实现了内存数据库的原型 --- LLOOGG Memory DB
- 2009 – Salvatore 发布了 Redis ，在接下来的十余年里， Redis 成为了业界最流行的 KV 缓存数据库
- 2020 - Salvatore 宣布退出 Redis 的维护，并把它移交给 Redis Labs （一家商业商业公司）
- 2021 - Redis Labs 改名为 Redis Ltd. 或者 Redis
- 2024 - Redis Ltd. 把 Redis 的许可证从开放的 BSD 协议修改成为不开放的 RSALv2 协议
- 2024 - 在 Linux Foundation 的支持下，成立 Valkey 社区。 Valkey 作为“开源版本的 Redis”，和 Redis 分道扬镳
- 2024 – Valkey 8.0 发布，开始支持 Valkey Over RDMA

Redis/Valkey Over RDMA

Redis/Valkey 的性能瓶颈

Redis/Valkey 历史上使用单线程模型，在 Intel Skylake 平台上使用 TCP 测试， 1KB 大小的 KV 场景下，具备大约 160K QPS 。 1 个 Master 和多个 Replica 的部署模式能一定程度缓解极端场景下的性能问题：



- 多 Replica 带来的成本问题
- 同步数据的延迟问题
- 极端场景下的缓存击穿问题

ByteDance字节跳动

Redis/Valkey 的热点函数分布

Children	Self	Command	Shared Object	Symbol
7.67%	7.67%	redis-server	[ip_tables]	[k] ipt_do_table
4.56%	4.56%	redis-server	[xt_tcpudp]	[k] tcp_mt
3.78%	3.78%	redis-server	[kernel.vmlinux]	[k] tcp_sendmsg_locked
3.61%	3.61%	redis-server	[kernel.vmlinux]	[k] _raw_spin_lock
2.56%	2.56%	redis-server	[kernel.vmlinux]	[k] copy_user_generic_unrolled
2.30%	2.30%	redis-server	[kernel.vmlinux]	[k] tcp_recvmsg
1.91%	1.91%	redis-server	[kernel.vmlinux]	[k] sock_poll
1.77%	1.77%	redis-server	[kernel.vmlinux]	[k] sock_rfree
1.73%	1.73%	redis-server	[kernel.vmlinux]	[k] ip_send_check
1.68%	1.68%	redis-server	[kernel.vmlinux]	[k] _raw_spin_lock_bh
1.45%	1.45%	redis-server	[kernel.vmlinux]	[k] __fget
1.42%	1.42%	redis-server	[kernel.vmlinux]	[k] tcp_cleanup_rbuf
1.39%	1.39%	redis-server	[kernel.vmlinux]	[k] tcp_poll
1.26%	1.26%	redis-server	[kernel.vmlinux]	[k] __tcp_transmit_skb
1.22%	1.22%	redis-server	[kernel.vmlinux]	[k] __check_object_size
1.19%	1.19%	redis-server	[kernel.vmlinux]	[k] tcp_write_xmit
1.16%	1.16%	redis-server	[kernel.vmlinux]	[k] __skb_clone

Redis/Valkey Over RDMA --- socket 和 RDMA 不同的编程模型

- TCP

```
while (ret = read(sockfd, buf, sizeof(buf)) > 0) {
    handle_input(buf, ret);
}
```

- RDMA

```
ctx->buf = memalign(page_size, size);
ctx->mr = ibv_reg_mr(ctx->pd, ctx->buf, size, access_flags);
struct ibv_sge list = {
        .addr   = (uintptr_t) ctx->buf, .length = ctx->size, .lkey   = ctx->mr->lkey
};
struct ibv_recv_wr wr = {
        .wr_id      = PINGPONG_RECV_WRID,
        .sg_list    = &list,
        .num_sge    = 1,
};
ret = ibv_post_recv(ctx->qp, &wr, &bad_wr);
```

Redis/Valkey Over RDMA --- 网络 I/O 模型抽象

```
typedef struct ConnectionType {
    int (*listen)(connListener *listener);
    connection *(*conn_create)(void);
    connection *(*conn_create_accepted)(int fd, void *priv);
    void (*shutdown)(struct connection *conn);
    void (*close)(struct connection *conn);
    int (*blocking_connect)(struct connection *conn, const char *addr, int port, long long timeout);
    int (*accept)(struct connection *conn, ConnectionCallbackFunc accept_handler);
    int (*write)(struct connection *conn, const void *data, size_t data_len);
    int (*writev)(struct connection *conn, const struct iovec *iov, int iovcnt);
    int (*read)(struct connection *conn, void *buf, size_t buf_len);
    int (*set_write_handler)(struct connection *conn, ConnectionCallbackFunc handler, int barrier);
    int (*set_read_handler)(struct connection *conn, ConnectionCallbackFunc handler);
    int (*has_pending_data)(void);
    int (*process_pending_data)(void);
    ...
} ConnectionType;
```

Redis/Valkey Over RDMA --- 事件驱动模型

- TCP

```
event.events = EPOLLIN | EPOLLOUT;
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &event);
```

- RDMA

```
ibv_post_recv(qp, &wr, &bad_wr);
ibv_post_send(qp, &send_wr, &bad_wr);
...
ibv_poll_cq(cq, 1, &wc);
switch (wc.opcode) {
    case IBV_WC_RECV: ...
    case IBV_WC_RECV_RDMA_WITH_IMM: ...
    case IBV_WC_SEND: ...
    ...
}
```

RDMA 编程没有 POLLOUT 事件，不能由外部触发 write_handler！

ByteDance字节跳动

Redis/Valkey Over RDMA --- EPOLLIN/EPOLLOUT 事件模拟

- EPOLLIN

1.ibv_comp_channel 可以被 EPOLLIN 事件唤醒，并处理 CQ 事件。
2.和 Remote Side 使用 RDMA WRITE WITH IMM 交换数据，收到了数据则产生 CQ 事件。
3.则 EPOLLIN 处理的过程中，可以覆盖“收到了数据”逻辑。

- EPOLLOUT

1.ibv_comp_channel 缺少 EPOLLOUT 事件唤醒。
2.在 RDMA 连接驱动中，使用链表存储所有的 Write Handler 作为 pending data 。
3.Redis/Valkey 在陷入事件等待循环前，尝试处理 pending data 。
4.在 RDMA 驱动中处理所有的 pending data ，尝试发送数据。

Redis/Valkey Over RDMA --- RDMA 在流式数据交换协议下的问题

Hello World 的协议报文：

*3\r\n$3\r\nSET\r\n$5\r\nHello\r\n$5\r\nWorld\r\n

- 不同的命令也有不同的长度。例如 PING 命令长度较短，但是 SET KEY VALUE 命令可长可短。
- 执行 GET KEY 命令，很难预测返回的 VALUE 有多少个字节。
- INFO 命令通常需要一次发送即可，返回的报文通常在很多个片段中。

如果使用 Receive buffer 来接收消息：

- 多大比较合适？例如 GET KEY ，无法预测即将返回的 VALUE 长度，不希望在 RDMA 场景下对消息的长度带来任何限制。
- 多少个 Receive buffer 比较合适呢？例如 INFO 命令可能会返回多个数据片段， COMMAND 命令可能返回数百个数据片段。
- 如果使用一个 Receive buffer 接收消息，在接收到消息后回复 ACK ，并请求接下来的数据，可以节省 Receive buffer ，但是带来了额外的 RTT 开销。

Redis/Valkey Over RDMA --- 传输协议设计

Redis/Valkey Over RDMA 传输协议整体上分成 2 部分：

- Control-Plane
使用固定大小 32 字节的消息（大端格式），用于交换“控制消息”。使用 IBV_WR_SEND 操作。

- Data-Plane
使用 IBV_WR_RDMA_WRITE （可选）和 IBV_WR_RDMA_WRITE_WITH_IMM （必选）交换业务数据。

Redis/Valkey Over RDMA --- 控制消息

- GetServerFeature： 客户端用于获取服务端的特性支持。
- SetClientFeature： 客户端用于设置特征支持（服务端的特性的子集）。
- CommandFoo： 未来可支持的可选命令。

GetServerFeature 和 SetClientFeature 允许在未来支持更多的功能和扩展。例如通过特性协商 FeatureFoo 决定是否支持 CommandFoo 。

- Keepalive： 简单的探活报文，用于发现 Remote Side 服务器崩溃等。

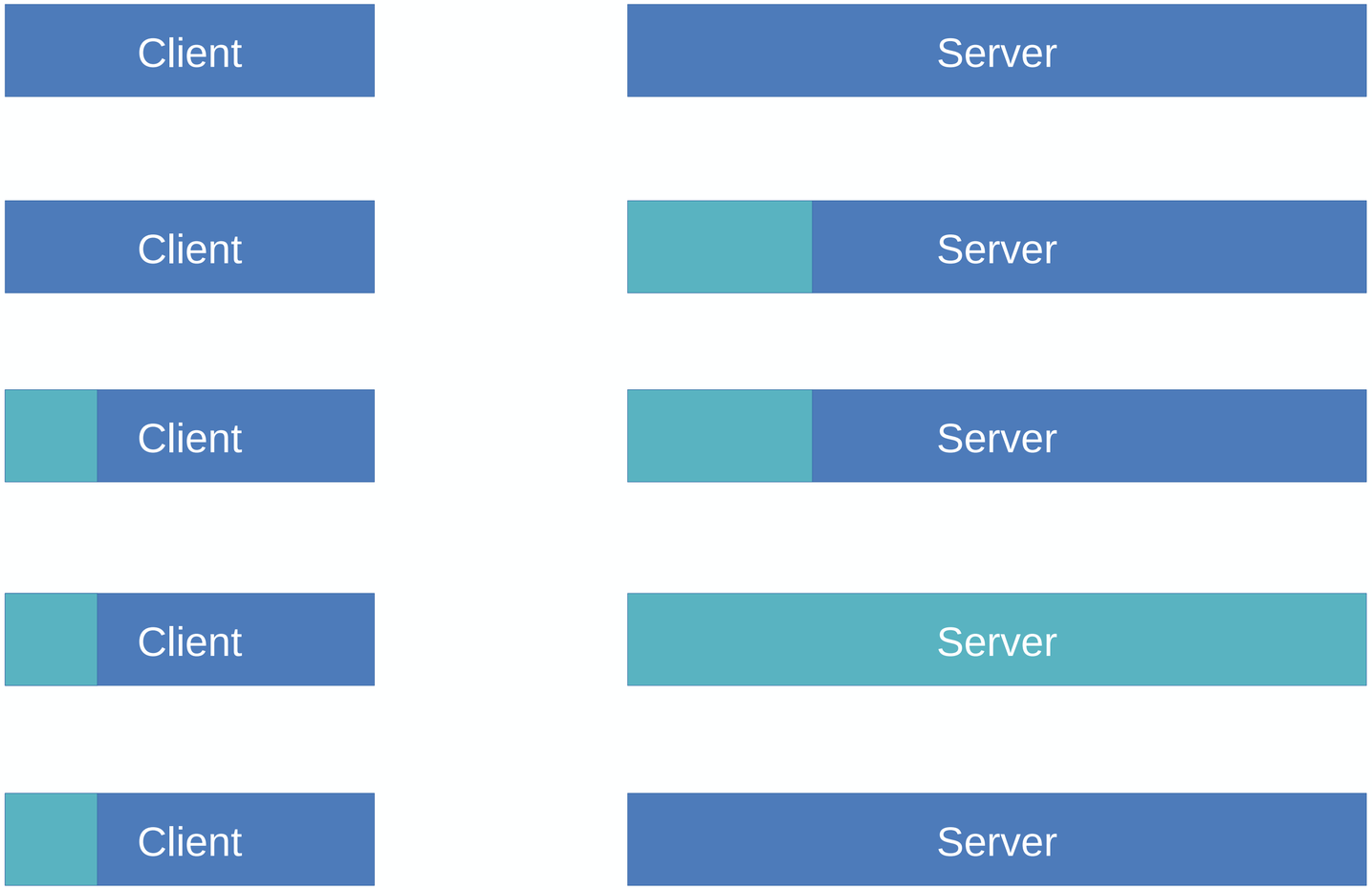
- RegisterXferMemory： 注册数据传输缓存。该缓存对于自身来说是“ RX transfer buffer“ ，对于 Remote Side 来说则是“ TX transfer buffer” 。

```
typedef struct ValkeyRdmaMemory {
        uint16_t opcode;
        uint8_t reserved[14];
        uint64_t addr;
        uint32_t length;
        uint32_t key;
} ValkeyRdmaMemory
```

ByteDance字节跳动

Redis/Valkey Over RDMA --- 业务数据交换

- 初始化：客户端使用 512K 、服务端使用 1M 的 RX transfer buffer 。处于可用状态。
- 客户端使用 RDMA WRITE WITH IMM 向服务端的 RX transfer buffer 写入数据。 IMMDT 为本次写入的数据长度。
- 同上，服务端回复数据。
- 服务端的 RX transfer buffer 全部写满，客户端等待服务端注册新的 RX transfer buffer 。
- 服务端注册新的 RX transfer buffer 。

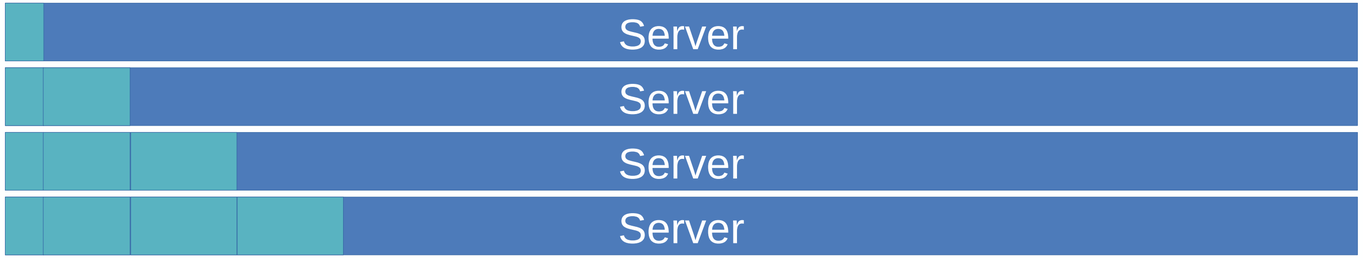


ByteDance字节跳动

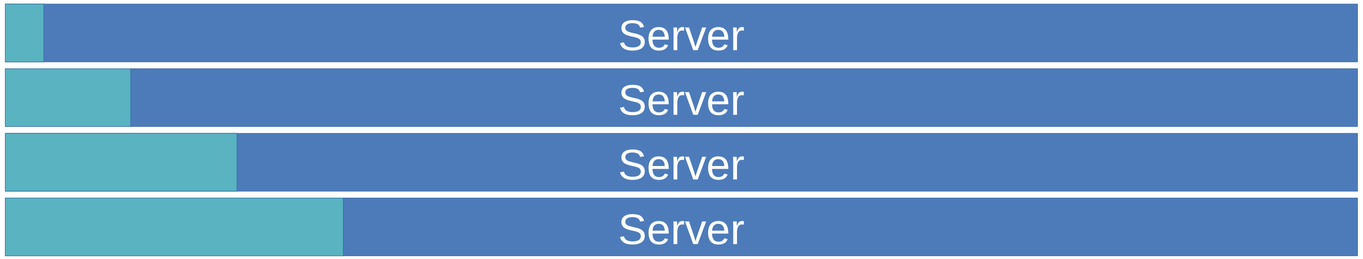
Redis/Valkey Over RDMA --- RDMA WRITE/RDMA WRITE WITH IMM

*3\r\n
$3\r\nSET\r\n
$5\r\nHello\r\n
$5\r\nWorld\r\n

RDMA WRITE WITH IMM(4)
RDMA WRITE WITH IMM(9)
RDMA WRITE WITH IMM(11)
RDMA WRITE WITH IMM(11)
Remote Side 将收到 4 次通知！



RDMA WRITE
RDMA WRITE
RDMA WRITE
RDMA WRITE WITH IMM(35)
Remote Side 将收到 1 次通知，类 writev 操作提供更好的性能！



ByteDance字节跳动

性能表现

性能表现

	Requests Per Second		Latency in us (avg/p95)	
	TCP	RDMA	TCP	RDMA
PING	214601.48	512794.22	132/191	56/87
SET	161595.27	267881.06	177/279	109/151
GET	179784.97	347789.78	157/215	83/111

- 服务端： ./src/valkey-server --port 6379 --loadmodule src/valkey-rdma.so port=6379 bind=* --loglevel verbose --protected-mode no --server_cpulist 12 --bio_cpulist 3 --aof_rewrite_cpulist 13 --bgsave_cpulist 13 --appendonly no
- 客户端： ./src/valkey-benchmark -h XX.XX.XX.XX -p 6379 -c 30 -n 10000000 --threads 4 -d 1024 -r 10000000 -t ping,set,get *--rdma*
- 测试硬件： Intel(R) Xeon(R) Platinum 8260/Mellanox ConnectX-5
- 测试软件： Debian GNU/Linux 9， Linux-5.4

ByteDance字节跳动

现状和规划

现状

- Valkey 8.0 开始正式支持 Valkey Over RDMA
  - ✓ 客户端和服务端互相连接支持 RDMA
  - ✗ 服务端多副本之间连接不支持 RDMA
  - ✗ Valkey Over RDMA 不支持 TLS
  - ✗ 在 KV 较大的场景下，额外的内存拷贝带来了性能退化。 Zero Copy 更好地 AI 场景下的 splitwise

- libvalkey 是官方 C/C++ 语言客户端，支持 Valkey Over RDMA 。其他语言的客户端不支持。

规划

- 通过 GLIDE 工程，实现 golang 、 java 、 nodejs 、 python 多语言支持。
- RDMA 实现子进程继承，允许多副本之间使用 RDMA 同步数据。
- 使用 RDMA 的 SRQ 降低服务端的资源消耗。
- RDMA transfer buffer 自适应大小，平衡性能和资源消耗。

THANKS

ByteDance字节跳动